Lean

PENG

October 30, 2026

Contents

1 Introduction **1**

2 Dependent Type Theory **1**

3 Tactics **2**

A Tools **2**

1 Introduction

Definition 1.1 (lean)**.** *Lean is an expression-oriented functional language based on dependent type theory.*

2 Dependent Type Theory

`https://lean-lang.org/theorem_proving_in_lean4/`
Every expression has an associated type.

Definition 2.1 (object)**.**

Definition 2.2 (types)**.**

```
Nat
Int
Float
String
Bool -- True or False
List
Type
```

Definition 2.3 (declare object)**.**

```
def obj : type := value -- define constant
#check obj -- check type
#eval expr -- evaluate
```

Definition 2.4 (proposition)**.**

Definition 2.5 (theorem)**.**

Definition 2.6 (function)**.**

```
def func (arg1 : type) ... (argn : type) : type :=
    body
```

3 Tactics

Tactic 3.1 (reflexivity of equality)**.** `ref` *proves goals of the form* `X = X`*.*

Tactic 3.2 (rewrite)**.** *If* `h` *is a proof of an equality* `X = Y`*, then* `rw [h]` *replace all* `X` *with* `Y`*.*

```
repeat rw [h] -- rewrite at once
rw [← h] -- turn Y into X
```

Definition 3.1 ($\mathbb{N}$)**.** $\mathbb{N}$ *is the natural numbers defined by* `0 : ℕ` *and* `succ (0 : ℕ) : ℕ`*.*

Theorem 3.1 (successor)**.** *The successor of* `n` *means the number after* `n`*.*

```
two_eq_succ_one -- proof of 2 = succ 1
add_succ x d : x + succ d = succ (x + d)
```

Theorem 3.2 (adding 0)**.** *Define* `a + 0` *to be* `a` *for any number* `a`*.*

```
add_zero   -- proof of ? + 0 = ?
add_zero x -- proof of x + 0 = x
```

A Tools

Definition A.1 (comment)**.**

```
/- comment blocks can be nested -/
-- line comment
```